S11 · Dos bases más y una caché

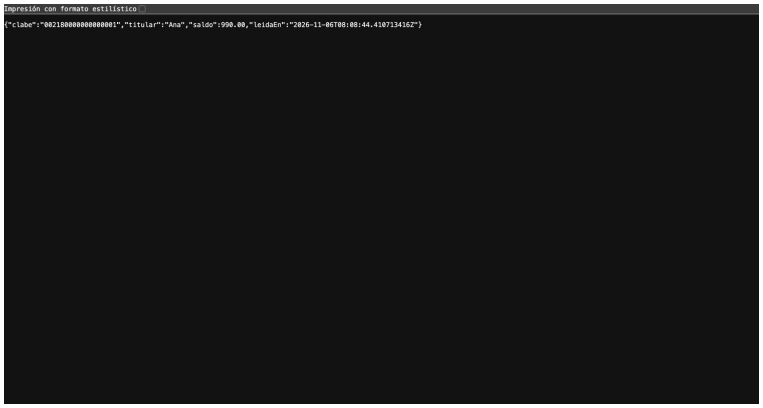
Pablo Vaquero · Cuenta 318259624 · Grupo 2
Sistemas Distribuidos · 6 de noviembre de 2026

PostgreSQL conserva el dinero. Redis acelera la consulta del saldo y MongoDB guarda avisos con formas distintas por canal. La autorización del cargo sigue consultando PostgreSQL, incluso si Redis muestra un saldo atrasado.

Parte 0 · Punto de partida

Prueba inicial	Resultado real
Diez GET del saldo	10 lecturas a PostgreSQL
Redis	0 hits, 0 misses
MongoDB	0 avisos
Transferencia base de $10	201; el aviso todavía aparece en PostgreSQL

Se partió de la rama oficial 06-poliglota-base, commit 9d10c2cdfce2d8d6d171d655bc680b6d2a1e23be. Se usaron las cuentas de Ana, Beto y Carla del enunciado. El proyecto Docker es mexibanco-s11-pablo, aislado de los labs previos, y el gateway de la prueba usa el puerto 18081.



Captura real del navegador: GET /api/cuentas/...001 antes de implementar la caché. Los diez cuerpos y el conteo están en 01-base.json.

Los horarios con Z de las respuestas están en UTC; las pruebas ocurrieron el 6 de noviembre en Ciudad de México. El código de este lab se distingue de la base oficial en los archivos de caché, la bitácora de avisos y el consumidor.

Parte 1 · Una caché compartida

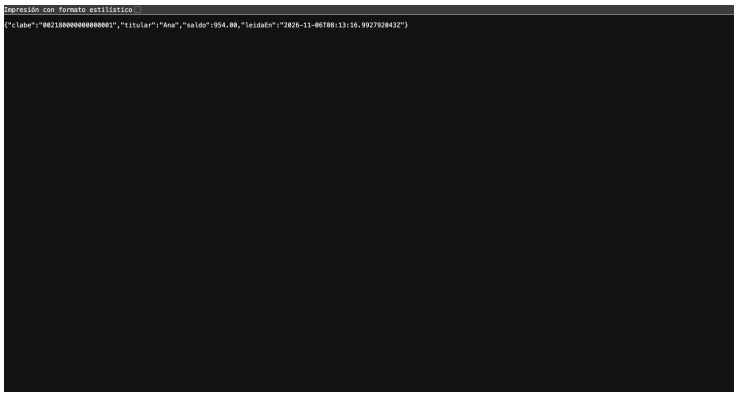
ConfiguracionCache activa @EnableCaching. La caché cuentas serializa CuentaVista con el ObjectMapper de Spring y Jackson2JsonRedisSerializer. Cada entrada tiene TTL de 30 segundos y transactionAware(): la invalidación espera al commit local.

```
@Cacheable(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
ConsultaDeSaldo.consultar(clabe)

@CacheEvict(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
CuentaService.cargar(...) / abonar(...)
```

ConsultaDeSaldo es una clase distinta para que las llamadas pasen por el proxy de Spring. CuentaService.consultar, que autoriza cargos, no tiene caché. No se reemplaza la verdad por la copia.

Comprobación	Resultado
Seis consultas consecutivas	3 instancias distintas; una sola lectura a PostgreSQL
leidaEn	2026-11-06T08:12:45.818169959Z
Clave	cuentas::002180000000000001
TTL observado	30 segundos
EXISTS tras transferir $1	0
Saldo tras el siguiente GET	978.0



Captura real de la consulta HTTP con la implementación activa. 02-cache.json conserva las seis respuestas, X-Instancia, el JSON de Redis, TTL e invalidación.

DEL después del commit evita anunciar un valor de una transacción que podría deshacerse. Aun así, una caché de lectura no ofrece consistencia fuerte: una lectura concurrente puede volver a guardar un dato atrasado. El TTL limita la vida de cada copia.

Mediciones · Aciertos y latencia

Escenario	Hits	Misses	Hit ratio	Lecturas PG
A: 100 GET	99	1	99.00%	1
B: 100 GET y 10 transferencias	100	20	83.33%	20

Cada escenario empezó borrando solo la clave de Ana y reiniciando contadores de Redis. En B hay 120 accesos: cien GET del cliente y veinte consultas HTTP de la saga (origen y destino). Las transferencias invalidan copias. Por eso el denominador no es cien y las lecturas de PostgreSQL no deben interpretarse como si solo hubiera tráfico del cliente.

Caché	Corrida	Media ms	Mediana ms	p95 ms	Lecturas PG
con	1	5.44	5.09	8.56	1
con	2	4.34	4.04	5.89	1
con	3	7.21	6.24	14.92	1
sin	1	7.34	6.37	13.12	100
sin	2	6.15	5.07	11.46	100
sin	3	4.96	4.59	8.09	100

Cada fila contiene cien peticiones secuenciales medidas en el cliente con perf_counter. Se conservan las seiscientas muestras. Hubo calentamiento antes de medir. Para desactivar caché se recrearon las tres réplicas con CACHE_TIPO=none; después se restauró Redis.

La latencia es ruidosa: una corrida sin caché fue más rápida que una con caché. No atribuyo toda diferencia a Redis ni generalizo este resultado a producción. El beneficio más claro aquí es capacidad: las lecturas a PostgreSQL bajaron de cien a una por corrida. Las ejecuciones ocurrieron en la misma laptop, con otros procesos activos.

Archivos: 03-hit-ratio-A.json, 03-hit-ratio-B.json y 04-latencia-{con,sin}-{1,2,3}.json. Las operaciones manuales GET de redis-cli se hicieron fuera de las ventanas de medición.

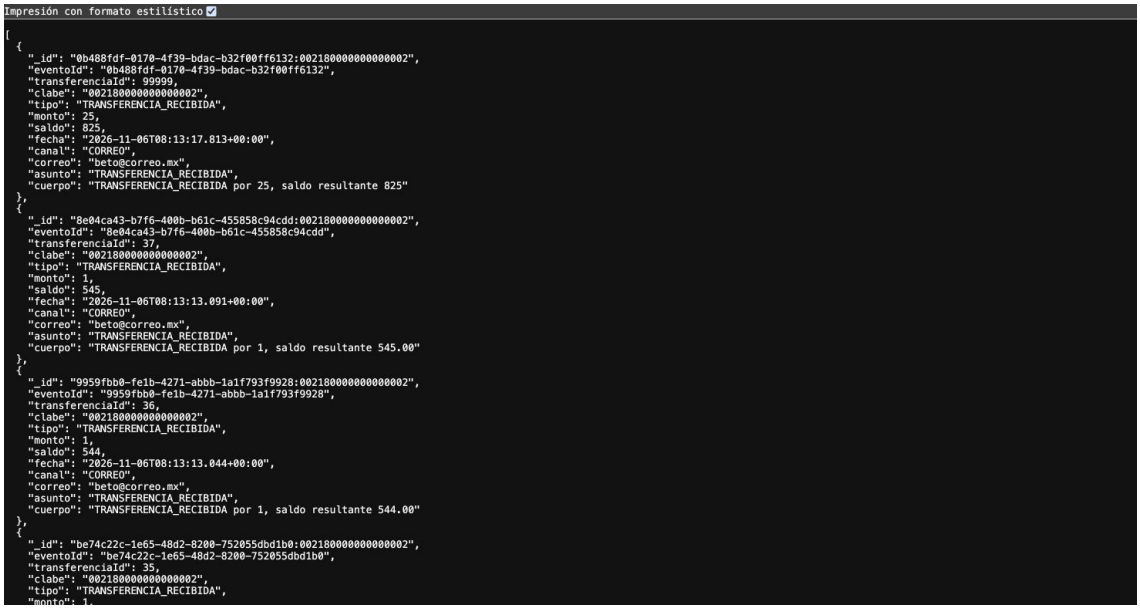
Parte 2 · Avisos en MongoDB

El consumidor escribe exclusivamente en MongoDB para estos eventos y hace ack después de guardar. La bandeja antigua de PostgreSQL se conserva para SPEI; el evento de transferencia ya no escribe en ambos motores.

Cada aviso tiene _id = eventoId:clabe, los campos comunes y solo los de su canal. Monto y saldo se guardan como Decimal128, y fecha como Date tomada del evento. Un registro por destinatario permite completar un evento parcialmente escrito sin duplicar el primero.

Canal	Campos propios
SMS	telefono, texto
CORREO	correo, asunto, cuerpo
PUSH	token, titulo, cuerpo

Se hicieron dos transferencias para recorrer los canales y doce pequeñas para comprobar el límite. El endpoint devolvió 10 avisos, del más reciente al más viejo. En el experimento se verificaron 28 avisos de esas catorce transferencias. La colección acumulaba 72 documentos por las pruebas anteriores de caché.



Captura real del endpoint /api/notificaciones/...002, mostrado con la vista JSON del navegador.

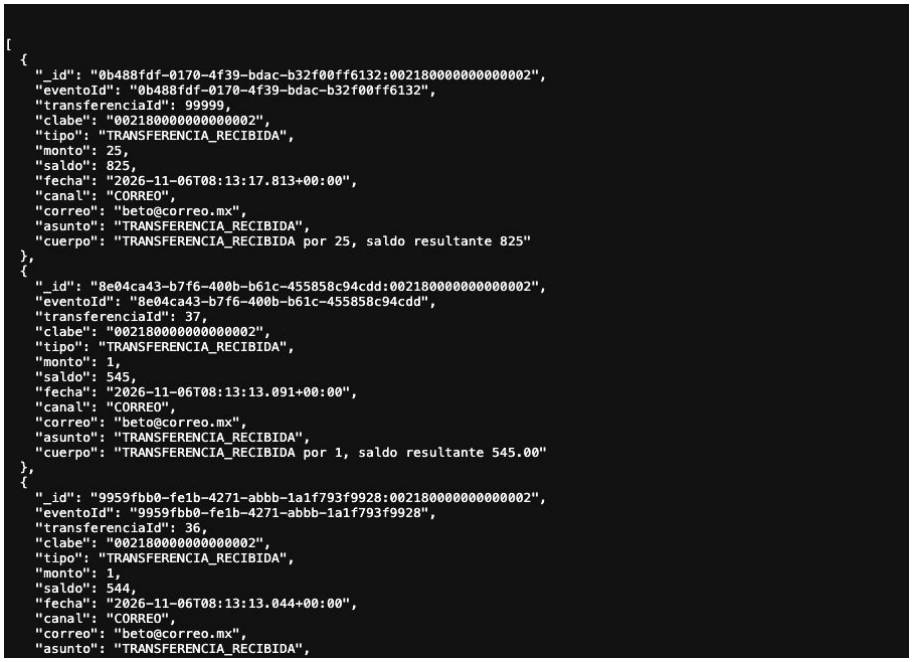
Índice y Parte 3 · Evento duplicado

Plan de consulta	Documentos examinados	Resultados
Índice clabe_fecha; IXSCAN	10	10
Recorrido natural; COLLSCAN y SORT	72	10

El índice es {clabe: 1, fecha: -1}. Filtra por CLABE y permite obtener los últimos diez en el orden requerido sin ordenar toda la colección. Se conservan los dos explain completos en 05-mongo.json; con esta colección pequeña no se demuestra una mejora estable de latencia.

Se publicó dos veces el evento 0b488fdf-0170-4f39-bdac-b32f00ff6132, con el mismo cuerpo y eventoId. Después de la primera publicación hubo dos avisos; después de la segunda siguió habiendo dos. Los logs dicen ya estaba para cada destinatario y cero avisos nuevos.

registrar inserta cada documento y solo atrapa DuplicateKeyException. Una falla real de MongoDB se propaga para reencolar, no se disfraza de duplicado. Si el proceso muere entre el primer y segundo destinatario, la siguiente entrega encuentra el primero y completa el segundo. Los dos documentos no son una transacción conjunta: pueden verse parcialmente durante la recuperación.



```
[
  {
    "_id": "0b488fdf-0170-4f39-bdac-b32f00ff6132:00218000000000002",
    "eventoId": "0b488fdf-0170-4f39-bdac-b32f00ff6132",
    "transferenciaId": 99999,
    "clabe": "00218000000000002",
    "tipo": "TRANSFERENCIA_RECIBIDA",
    "monto": 25,
    "saldo": 825,
    "fecha": "2026-11-06T08:13:17.813+00:00",
    "canal": "CORREO",
    "correo": "beto@correo.mx",
    "asunto": "TRANSFERENCIA_RECIBIDA",
    "cuerpo": "TRANSFERENCIA_RECIBIDA por 25, saldo resultante 825"
  },
  {
    "_id": "8e04ca43-b7f6-400b-b61c-455858c94cdd:00218000000000002",
    "eventoId": "8e04ca43-b7f6-400b-b61c-455858c94cdd",
    "transferenciaId": 37,
    "clabe": "00218000000000002",
    "tipo": "TRANSFERENCIA_RECIBIDA",
    "monto": 1,
    "saldo": 545,
    "fecha": "2026-11-06T08:13:13.091+00:00",
    "canal": "CORREO",
    "correo": "beto@correo.mx",
    "asunto": "TRANSFERENCIA_RECIBIDA",
    "cuerpo": "TRANSFERENCIA_RECIBIDA por 1, saldo resultante 545.00"
  },
  {
    "_id": "9959fbb0-fe1b-4271-abbb-1a1f793f9928:00218000000000002",
    "eventoId": "9959fbb0-fe1b-4271-abbb-1a1f793f9928",
    "transferenciaId": 36,
    "clabe": "00218000000000002",
    "tipo": "TRANSFERENCIA_RECIBIDA",
    "monto": 1,
    "saldo": 544,
    "fecha": "2026-11-06T08:13:13.044+00:00",
    "canal": "CORREO",
    "correo": "beto@correo.mx",
    "asunto": "TRANSFERENCIA_RECIBIDA",
```

Captura real del aviso publicado en la prueba, visible una sola vez en los resultados. La comprobación de ambos conteos y los logs están en 06-duplicado.json.

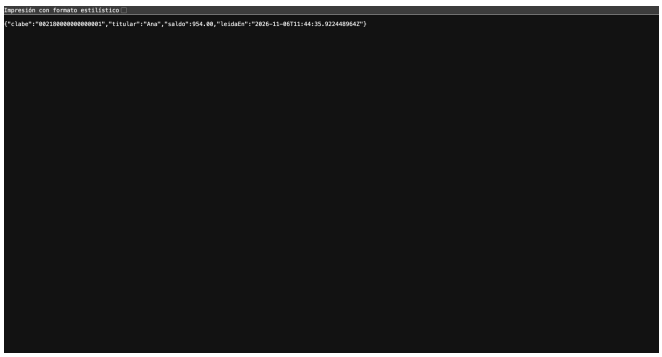
Parte 4 · La copia puede mentir

Se llenó Redis con el saldo real de Ana, $954.00, y se cambió PostgreSQL directamente a $5. Este UPDATE intencional evitó @CacheEvict y solo se hizo sobre las cuentas del laboratorio. El GET todavía mostró el saldo anterior.

Tiempo desde el primer control	TTL	Saldo mostrado
0.1 s	28	954.0
5.3 s	23	954.0
10.6 s	18	954.0
15.8 s	13	954.0
21.1 s	7	954.0
26.2 s	2	954.0
31.6 s	30	5.0

El refresco se detectó entre el control de 26.2 s y el de 31.6 s. El muestreo fue aproximadamente cada cinco segundos: 31.6 s es el momento de detección, no una medición exacta de la vida de la entrada. Al refrescar, el TTL volvió a 30 porque se guardó una copia nueva.

Mientras la caché decía $954.00, se intentó transferir $100. La respuesta fue HTTP 422: saldo insuficiente. PostgreSQL conservó $5. La autorización consultó la verdad, no el saldo que se mostraba.



Captura real del GET durante el experimento: la copia aún muestra $954.00. 07-dato-viejo.json conserva la respuesta 422, el TTL y el seguimiento.

Al terminar se restauró el saldo previo y se borró la copia de Redis. El UPDATE de la demo rompe temporalmente el cuadre saldo-ledger: no es un procedimiento de operación real.

Preguntas de cierre y validación

1 · ¿Por qué MongoDB para avisos y no para saldos?

Los avisos se insertan y se leen completos por CLABE, y cada canal tiene campos propios. MongoDB permite guardar esas formas sin columnas vacías. El saldo necesita validación concurrente, idempotencia y la transacción local que ya están en PostgreSQL. Cambiarlo de motor exigiría demostrar de nuevo esas garantías; separar por moda no las mejora.

2 · ¿Qué compró la caché?

A dio 99 % y B 83.33 %. Las escrituras invalidan y la saga añade consultas a ambas cuentas. Las latencias se superponen entre corridas; la reducción de cien lecturas a una es la evidencia más clara de capacidad ahorrada en PostgreSQL. El costo es una copia con edad y otro servicio que operar.

3 · ¿Qué pasa si se autoriza desde caché?

Podría aceptarse un cargo de $100 cuando la verdad solo tiene $5, como mostró la Parte 4. El saldo para mostrar puede tolerar atraso; el que decide si se mueve dinero no debe hacerlo. La API de cargo vuelve a validar contra PostgreSQL y rechazó la operación.

4 · ¿Por qué DEL y después del commit?

Borrar deja que la próxima lectura recupere la verdad; hacer SET con un saldo calculado antes puede sobrescribir una versión más reciente. Invalidar antes del commit permitiría que otra consulta repueble la copia con el valor aún no confirmado. transactionAware retrasa el DEL hasta después de confirmar, aunque no elimina todas las carreras con lectores concurrentes.

Pruebas y límites

Diez pruebas de cuenta y diecinueve de notificación aprobadas. Incluyen ack después de persistir, datos inválidos a DLQ, reencolar ante falla real y completar el segundo aviso tras encontrar el primero duplicado. Los experimentos además usan Redis, MongoDB y PostgreSQL reales. No se hizo el reto opcional de Redis caído: en esta versión una falla de caché puede afectar las peticiones.

Fuente: github.com/OscarRuiz21/sd-2027-1/blob/main/labs/s11-poliglota/README.md. Entrega individual preparada con apoyo de Codex; datos, capturas y salidas provienen de la ejecución local.